2026-05-07-p2-f23-cline-browser-tool

P2-F23 — Cline Browser Tool Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (`- [ ]`) syntax for tracking.

> **Programme position:** F23 is the **third** Phase 2 feature of CLI-Agent Fusion (after F21 Codex Approval Modes and F22 Aider Git Auto-Commit). Task T01 advances PROGRESS.md from "Phase 2: F22 closed; F23 next candidate (brainstorm)" to "Phase 2 of CLI-Agent Fusion programme: F23 (Cline Browser Tool) in flight" and appends an F23 evidence header to `docs/improvements/07_phase_2_evidence.md` (already created in F21-T01, extended by F22-T01).

Goal: Ship a real, end-to-end **6-tool browser-automation suite** for the HelixCode CLI agent, modelled on cline's Puppeteer surface. F23 EXTENDS the existing `internal/tools/browser/` package (which already carries the chromedp infrastructure — `Controller`, `ActionExecutor`, `ScreenshotCapture`, `ConsoleMonitor`, `ChromeDiscovery`, `ScreenshotAnnotator` — at ~3 K LOC) by ADDING a thinner cline-style session façade (`BrowserSession`, `BrowserManager` with atomic-pointer lifecycle, `BrowserOptions` from env, `Snapshot`/`ScreenshotResult` value types, sentinel errors, `EnvVarHeadedMode` constant) and SIX new Tool-interface implementations (`browser_navigate`, `browser_snapshot`, `browser_click`, `browser_type`, `browser_screenshot`, `browser_close`) — each with the F21-mandated `RequiresApproval()` level mapped per spec §3.6 (`navigate`/`click`/`type`/`close` → `LevelEdit`; `snapshot`/`screenshot` → `LevelReadOnly`). Adds a `/browser` slash command (`status` / `navigate <url>` / `close`); NO cobra subcommand. Headless by default; `HELIXCODE_BROWSER_HEADED=true` (case-insensitive literal `true`) enables headed mode. Screenshots written to per-session tempdir under `$XDG_DATA_HOME/helixcode/browser/screenshots/<session-id>/<n>.png` (mode `0600`); tool result returns the absolute file path (NOT base64). Idempotent close via `sync.Once`. Lifecycle: lazy-create on first navigate; explicit close via tool or slash; defensive `defer mgr.CloseSession()` at process exit.

Architecture: New files under `helix_code/internal/tools/browser/` — `types.go` (`Snapshot`, `ScreenshotResult`, `ManagerStatus`, sentinels `ErrNoActiveSession`/`ErrChromiumNotFound`/`ErrNavigationTimeout`/`ErrSelectorNotFound`/`ErrScreenshotTooLarge`, `EnvVarHeadedMode`, `MaxSnapshotBytes`, `MaxScreenshotBytes`), `options.go` (`BrowserOptions` + `OptionsFromEnv()`), `session.go` (`BrowserSession` with chromedp Context + cancel + per-session screenshot tempdir + atomic counter + `sync.Once` close), `manager.go` (`BrowserManager` with `atomic.Pointer[BrowserSession] current`, `EnsureSession`/`RequireSession`/`CloseSession`/`Status`, mutex serialises only lifecycle transitions), `navigate_tool.go` / `snapshot_tool.go` / `click_type_tools.go` / `screenshot_tool.go` / `close_tool.go` (six Tool impls), `register.go` (single `RegisterAll(reg, mgr)` entry point). New `internal/commands/browser_command.go` for the `/browser` slash. Two existing files get small

additions: `cmd/cli/main.go` (1) construct `mgr := browser.NewBrowserManager(browser.NewDefaultChromeDiscovery(), c.logger)`, (2) call `browser.RegisterAll(c.toolRegistry, mgr)`, (3) register `commands.NewBrowserCommand(mgr)` slash; `internal/commands/registry.go` gets one new `Register(...)` call site. Coexists with the existing `BrowserLaunchTool`/`BrowserNavigateTool`/`BrowserScreenshotTool`/`BrowserCloseTool` in `internal/tools/browser_tools.go` (multi-browser API stays — back-compat). T09 picks the registration strategy: F23 tools register under `browser_navigate`/etc. names, with the existing ones renamed `_legacy` if any test needs to disambiguate.

Tech Stack: Go 1.26, testify v1.11, zap (already direct), chromedp v0.15.1 (ALREADY direct in `helix_code/go.mod`), cdproto v0.0.0-20260405000525-47a8ff65b46a (ALREADY direct). **Zero new external deps** (`os`, `sync`, `sync/atomic`, `time`, `errors`, `fmt`, `strings`, `path/filepath`, `context`, `image/png`, `net/http/httptest` all stdlib). `go mod tidy` after T09 must produce no diff in either `go.mod` or `go.sum`. T10's verification step asserts this loudly.

Spec: `docs/superpowers/specs/2026-05-07-p2-f23-cline-browser-tool-design.md` (commit `83d401d`).

Working directory for `go` commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (full 4-term applied to F23 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/tools/browser internal/commands/browser_command.go &&
        echo BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — F23 can degenerate in five ways: (a) `browser_navigate` reports success but the page is still on `about:blank` (the `chromedp.Navigate` action did not error but the page never loaded); (b) `browser_snapshot` returns empty/near-empty HTML because the snapshot read the DOM before `WaitReady("body")` resolved; (c) `browser_click` reports success but the DOM didn't mutate (the click matched a hidden/disabled element OR fired async without a settle wait); (d) `browser_screenshot` writes a 0-byte file or non-PNG bytes (chromedp returned `nil` byte buf and the implementation `os.WriteFile`-d it without checking); (e) `browser_close` flips `manager.current` to nil but never calls `cancel()`, leaving the chromium subprocess running. The seven "what counts as the browser tool works" criteria — (1) `browser_navigate` + `browser_snapshot(html)` against a real `httptest.Server` returns content containing the fixture sentinel `FIXTURE_LOADED_42` AND `len > 100` (PHASE-A); (2) `browser_snapshot(text)` returns the same fixture sentinel in plain text (PHASE-B); (3) pre-click snapshot has `UNCLICKED`; click `#b`; post-click snapshot has `CLICKED_42` (positive byte differential — PHASE-C); (4) `browser_type("#in", "HELIX_42")` + post-snapshot has `value="HELIX_42"` or JS-eval reads back `HELIX_42` (PHASE-D); (5) `browser_screenshot` returns a path whose file exists, is `> 1024` bytes, and starts with PNG magic `89 50 4E 47 0D 0A 1A 0A`, AND `image/png.DecodeConfig` succeeds (PHASE-E); (6) `browser_close` then `browser_snapshot` returns `errors.Is(err, ErrNoActiveSession)` (PHASE-F); (7) 5 concurrent `EnsureSession` goroutines all receive the same `*BrowserSession` pointer (PHASE-G) — are each tested with both unit assertions AND a Challenge phase. The Challenge harness uses positive evidence: HTTP-server-served sentinel string equality (PHASE-

A/B), DOM-mutation byte differential (PHASE-C), input-value byte assertion (PHASE-D), PNG-magic + size + DecodeConfig (PHASE-E), `errors.Is` post-close (PHASE-F), pointer equality (PHASE-G). Byte-evidence mismatch is a hard Challenge failure. Skip is permitted ONLY when chromium is absent and MUST emit `SKIP-OK: #P2-F23 chromium not available`. Absence-of-error is NEVER acceptable.

Why this is consequential: the browser tool is the cline-pattern crown jewel — without it, the CLI agent can't observe a running web UI, can't validate JS-heavy SPAs an aider/codex agent edited, and can't run end-to-end browser-driven Challenges. F21 (approval) gates the user surface; F22 (autocommit) makes edits visible; F23 makes the agent's eyes work. F23's discriminating tests are: (i) PHASE-A's fixture-sentinel-in-snapshot (proves real chromium load); (ii) PHASE-C's CLICKED_42 byte differential (proves real DOM mutation); (iii) PHASE-D's input-value byte readback (proves real keystroke delivery); (iv) PHASE-E's PNG-magic + DecodeConfig success (proves real PNG bytes, not nil-buf-written-to-disk); (v) PHASE-F's `ErrNoActiveSession` post-close (proves real teardown, not just status flip); (vi) PHASE-G's same-pointer concurrency (proves single chromium subprocess across concurrent calls). All six must produce positive evidence; none can be satisfied by absence-of-error.

Task list

- [ ] P2-F23-T01 — bootstrap F23 evidence section + advance PROGRESS to F23
- [ ] P2-F23-T02 — `internal/tools/browser/types.go` + `options.go`: BrowserOptions + Snapshot + ScreenshotResult + ManagerStatus + sentinels + EnvVarHeadedMode + MaxSnapshotBytes + MaxScreenshotBytes (TDD)
- [ ] P2-F23-T03 — `internal/tools/browser/manager.go` + `session.go`: BrowserManager with atomic-pointer lifecycle + BrowserSession with chromedp Context + per-session tempdir + sync.Once close + headed/headless option (TDD with stub chromedp seam)
- [ ] P2-F23-T04 — `internal/tools/browser/navigate_tool.go`: browser_navigate Tool impl with WaitReady + lazy session-create (TDD)
- [ ] P2-F23-T05 — `internal/tools/browser/snapshot_tool.go`: browser_snapshot Tool impl with html/text mode + 64 KB cap + Truncated flag (TDD)
- [ ] P2-F23-T06 — `internal/tools/browser/click_type_tools.go`: browser_click + browser_type Tool impls with NodeVisible + click-settle wait + selector-not-found mapping (TDD)
- [ ] P2-F23-T07 — `internal/tools/browser/screenshot_tool.go`: browser_screenshot Tool impl with PNG-magic verification + size > 1024 + image/png.DecodeConfig (TDD)
- [ ] P2-F23-T08 — `internal/tools/browser/close_tool.go`: browser_close Tool impl with idempotent-close semantics (TDD)

P2-F23-T09 — `internal/commands/browser_command.go`: /browser slash (status/navigate/close) + main.go wiring + browser.RegisterAll + integration test (gated on real chromium)

P2-F23-T10 — Challenge harness (7 phases A-G: navigate-and-snapshot + snapshot-text + click-mutates-DOM + type-into-input + screenshot-PNG-magic + close-tears-down + concurrent-session-sharing) gated on chromium + close-out + push 4 remotes non-force

---

Task 1: Bootstrap F23 evidence

Append F23 section header to `docs/improvements/07_phase_2_evidence.md` (created in F21-T01, extended by F22-T01) with spec SHA `83d401d`. Update PROGRESS.md current focus from "Phase 2 of CLI-Agent Fusion programme: F22 closed; F23 next candidate (brainstorm)" to "Phase 2 of CLI-Agent Fusion programme: F23 (Cline Browser Tool) in flight". Insert F23 task list (10 items). Verify zero new external deps:

```
cd HelixCode && grep -E "chromedp|cdproto" go.mod
# Expected: chromedp v0.15.1 + cdproto pinned (already present
        from earlier work).
git diff go.mod | grep -E "^\+|^-" | grep -v "^+++\|^---" &&
        echo "UNEXPECTED" || echo "clean"
```

Update `docs/CONTINUATION.md` root-level mid-flight section with F23 in-flight status (will be updated per task by T02-T10 commits).

Commit: `docs(P2-F23-T01): bootstrap F23 evidence + advance PROGRESS to F23 (Cline Browser Tool).`

Task 2: types.go + options.go (TDD)

Files: new `helix_code/internal/tools/browser/types.go`, new `helix_code/internal/tools/browser/types_test.go`, new `helix_code/internal/tools/browser/options.go`, new `helix_code/internal/tools/browser/options_test.go`.

Define in `types.go`: - `Snapshot struct { URL, Title, Mode, Content string; Truncated bool }`. - `ScreenshotResult struct { Path string; Bytes int64; Width, Height int }`. - `ManagerStatus struct { Active bool; ChromiumPath, ScreenshotDir string; Headed bool; CreatedAt time.Time }`. - Constants: `EnvVarHeadedMode = "HELIXCODE_BROWSER_HEADED"`, `MaxSnapshotBytes = 64 * 1024`, `MaxScreenshotBytes int64 = 5 * 1024 * 1024`. - Sentinel errors: `ErrNoActiveSession`, `ErrChromiumNotFound`, `ErrNavigationTimeout`, `ErrSelectorNotFound`, `ErrScreenshotTooLarge`.

Define in `options.go`: - `BrowserOptions struct { Headless bool; ViewportWidth, ViewportHeight int; NavigateTimeout, ClickWaitDuration time.Duration; ScreenshotMaxBytes int64 }`. - `OptionsFromEnv() BrowserOptions` — reads `HELIXCODE_BROWSER_HEADED` and

treats case-insensitive `"true"` as headed (Headless=false); everything else (including unset, `"True123"`, `"yes"`, `"1"`) is headless (Headless=true).

Failing tests FIRST:

```
func TestEnvVarHeadedMode_Pin(t *testing.T) {
    require.Equal(t, "HELIXCODE_BROWSER_HEADED",
        EnvVarHeadedMode)
}

func TestMaxSnapshotBytes_Pin(t *testing.T) {
    require.Equal(t, 64*1024, MaxSnapshotBytes)
}

func TestMaxScreenshotBytes_Pin(t *testing.T) {
    require.Equal(t, int64(5*1024*1024), MaxScreenshotBytes)
}

func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{
        ErrNoActiveSession, ErrChromiumNotFound,
        ErrNavigationTimeout,
        ErrSelectorNotFound, ErrScreenshotTooLarge,
    } {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}

func TestOptionsFromEnv_Default_Headless(t *testing.T) {
    t.Setenv(EnvVarHeadedMode, "")
    o := OptionsFromEnv()
    require.True(t, o.Headless)
}

func TestOptionsFromEnv_TrueLowercase_Headed(t *testing.T) {
    t.Setenv(EnvVarHeadedMode, "true")
    o := OptionsFromEnv()
    require.False(t, o.Headless)
}

func TestOptionsFromEnv_TrueUppercase_Headed(t *testing.T) {
    t.Setenv(EnvVarHeadedMode, "TRUE")
    o := OptionsFromEnv()
    require.False(t, o.Headless)
}

func TestOptionsFromEnv_NonBool_Headless(t *testing.T) {
```

```
    for _, v := range []string{"yes", "1", "True123",
        "headless", "0"} {
        t.Setenv(EnvVarHeadedMode, v)
        o := OptionsFromEnv()
        require.True(t, o.Headless, "value %q should be
        headless", v)
    }
}

func TestOptionsFromEnv_DefaultViewport(t *testing.T) {
    o := OptionsFromEnv()
    require.Equal(t, 1280, o.ViewportWidth)
    require.Equal(t, 720, o.ViewportHeight)
    require.Equal(t, 30*time.Second, o.NavigateTimeout)
    require.Equal(t, 500*time.Millisecond, o.ClickWaitDuration)
    require.Equal(t, MaxScreenshotBytes, o.ScreenshotMaxBytes)
}
```

Subject: `feat(P2-F23-T02): browser types + options - Snapshot/ScreenshotResult/ManagerStatus + sentinels + EnvVarHeadedMode (TDD).`

Task 3: manager.go + session.go (TDD)

Files: new `helix_code/internal/tools/browser/manager.go`, new `helix_code/internal/tools/browser/manager_test.go`, new `helix_code/internal/tools/browser/session.go`, new `helix_code/internal/tools/browser/session_test.go`.

session.go:

```
type BrowserSession struct {
    ctx             context.Context
    cancel          context.CancelFunc
    screenshotDir   string
    screenshotCount atomic.Uint64
    chromiumPath    string
    headed          bool
    createdAt       time.Time
    closeOnce       sync.Once
    log             *zap.Logger
}

func (s *BrowserSession) Run(ctx context.Context,
        actions ...chromedp.Action) error {
    if s == nil || s.ctx == nil {
        return ErrNoActiveSession
    }
```

```
    return chromedp.Run(s.ctx, actions...)
}

func (s *BrowserSession) NextScreenshotPath() string {
    n := s.screenshotCount.Add(1)
    return filepath.Join(s.screenshotDir, fmt.Sprintf("%d.png",
        n))
}

func (s *BrowserSession) Close() error {
    var rmErr error
    s.closeOnce.Do(func() {
        if s.cancel != nil { s.cancel() }
        if s.screenshotDir != "" {
            rmErr = os.RemoveAll(s.screenshotDir)
        }
    })
    return rmErr
}
```

manager.go:

```
type BrowserManager struct {
    current        atomic.Pointer[BrowserSession]
    screenshotRoot string
    discovery      ChromeDiscovery
    log            *zap.Logger
    mu             sync.Mutex
    sessionFactory func(ctx context.Context, mgr
        *BrowserManager, opts BrowserOptions) (*BrowserSession,
        error) // test seam
}

func NewBrowserManager(d ChromeDiscovery, log *zap.Logger)
        *BrowserManager {
    return &BrowserManager{
        screenshotRoot: defaultScreenshotRoot(), //
        $XDG_DATA_HOME/helixcode/browser/screenshots
        discovery:      d,
        log:            log,
        sessionFactory: defaultSessionFactory,
    }
}

func (m *BrowserManager) EnsureSession(ctx context.Context)
        (*BrowserSession, error) {
    if s := m.current.Load(); s != nil { return s, nil }
    m.mu.Lock()
    defer m.mu.Unlock()
```

```
    if s := m.current.Load(); s != nil { return s, nil } //
        double-check
    opts := OptionsFromEnv()
    s, err := m.sessionFactory(ctx, m, opts)
    if err != nil { return nil, err }
    m.current.Store(s)
    return s, nil
}

func (m *BrowserManager) RequireSession() (*BrowserSession,
        error) {
    s := m.current.Load()
    if s == nil { return nil, ErrNoActiveSession }
    return s, nil
}

func (m *BrowserManager) CloseSession() error {
    m.mu.Lock()
    defer m.mu.Unlock()
    s := m.current.Swap(nil)
    if s == nil { return nil }
    return s.Close()
}

func (m *BrowserManager) Status() ManagerStatus {
    s := m.current.Load()
    if s == nil { return ManagerStatus{Active: false, Headed:
        false} }
    return ManagerStatus{
        Active: true, ChromiumPath: s.chromiumPath,
        ScreenshotDir: s.screenshotDir, Headed: s.headed,
        CreatedAt: s.createdAt,
    }
}
```

Failing tests FIRST (uses a stub `sessionFactory` to avoid spawning real chromium in unit tests):

```
func newStubManager(t *testing.T) *BrowserManager {
    m := NewBrowserManager(nil, zap.NewNop())
    m.sessionFactory = func(_ context.Context, mgr
        *BrowserManager, _ BrowserOptions) (*BrowserSession,
        error) {
        return &BrowserSession{
            ctx: context.Background(), cancel: func() {},
            screenshotDir: t.TempDir(), chromiumPath: "/fake/
        chromium",
            createdAt: time.Now(), log: zap.NewNop(),
        }, nil
```

```
    }
    return m
}

func TestManager_EnsureSession_LazyCreates(t *testing.T) {
    m := newStubManager(t)
    require.Nil(t, m.current.Load())
    s, err := m.EnsureSession(context.Background())
    require.NoError(t, err); require.NotNil(t, s)
    require.Equal(t, s, m.current.Load())
}

func TestManager_EnsureSession_DoubleCallSamePointer(t
        *testing.T) {
    m := newStubManager(t)
    s1, _ := m.EnsureSession(context.Background())
    s2, _ := m.EnsureSession(context.Background())
    require.Same(t, s1, s2)
}

func TestManager_RequireSession_NilReturnsErr(t *testing.T) {
    m := newStubManager(t)
    _, err := m.RequireSession()
    require.ErrorIs(t, err, ErrNoActiveSession)
}

func TestManager_CloseSession_FollowedByRequireFails(t
        *testing.T) {
    m := newStubManager(t)
    _, _ = m.EnsureSession(context.Background())
    require.NoError(t, m.CloseSession())
    _, err := m.RequireSession()
    require.ErrorIs(t, err, ErrNoActiveSession)
}

func TestManager_CloseSession_Idempotent(t *testing.T) {
    m := newStubManager(t)
    _, _ = m.EnsureSession(context.Background())
    require.NoError(t, m.CloseSession())
    require.NoError(t, m.CloseSession()) // second close is a no-
        op
}

func TestManager_Concurrent_EnsureSession_SamePointer(t
        *testing.T) {
    m := newStubManager(t)
    const N = 10
    var wg sync.WaitGroup
```

```
    seen := make([]*BrowserSession, N)
    for i := 0; i < N; i++ {
        wg.Add(1)
        go func(idx int) {
            defer wg.Done()
            s, _ := m.EnsureSession(context.Background())
            seen[idx] = s
        }(i)
    }
    wg.Wait()
    for i := 1; i < N; i++ { require.Same(t, seen[0], seen[i]) }
}

func TestManager_Status_NoActiveSession(t *testing.T) {
    m := newStubManager(t)
    st := m.Status()
    require.False(t, st.Active)
}

func TestManager_Status_ActiveSession(t *testing.T) {
    m := newStubManager(t)
    _, _ = m.EnsureSession(context.Background())
    st := m.Status()
    require.True(t, st.Active)
    require.Equal(t, "/fake/chromium", st.ChromiumPath)
}

func TestSession_NextScreenshotPath_Monotonic(t *testing.T) {
    s := &BrowserSession{screenshotDir: t.TempDir()}
    p1 := s.NextScreenshotPath()
    p2 := s.NextScreenshotPath()
    require.NotEqual(t, p1, p2)
    require.Contains(t, p1, "1.png")
    require.Contains(t, p2, "2.png")
}

func TestSession_Close_Idempotent_SyncOnce(t *testing.T) {
    n := 0
    s := &BrowserSession{cancel: func() { n++ }, screenshotDir:
        t.TempDir()}
    require.NoError(t, s.Close())
    require.NoError(t, s.Close())
    require.Equal(t, 1, n) // cancel called exactly once
}

func TestSession_Run_NilCtx_ErrNoActiveSession(t *testing.T) {
    s := &BrowserSession{}
    err := s.Run(context.Background())
```

```
    require.ErrorIs(t, err, ErrNoActiveSession)
}
```

Non-obvious call: the `sessionFactory` is a test seam. Production wiring uses `defaultSessionFactory` (constructs a real chromedp Allocator + Context); unit tests override to avoid spawning chromium. Integration tests use the production factory and gate on chromium availability (skip-OK if not on PATH).

Subject: `feat(P2-F23-T03): browser manager + session - atomic-pointer lifecycle + sync.Once close + sessionFactory seam (TDD).`

Task 4: navigate_tool.go (TDD)

Files: new `helix_code/internal/tools/browser/navigate_tool.go`, new `helix_code/internal/tools/browser/navigate_tool_test.go`.

```
type BrowserNavigateToolV2 struct { mgr *BrowserManager; opts
        BrowserOptions }

func NewBrowserNavigateTool(mgr *BrowserManager, opts
        BrowserOptions) *BrowserNavigateToolV2

func (t *BrowserNavigateToolV2) Name() string { return
        "browser_navigate" }
func (t *BrowserNavigateToolV2) RequiresApproval()
        approval.ApprovalLevel { return approval.LevelEdit }
func (t *BrowserNavigateToolV2) Description() string { return
        "Navigate the browser session to a URL." }
func (t *BrowserNavigateToolV2) Category() tools.ToolCategory {
        return tools.CategoryBrowser }

func (t *BrowserNavigateToolV2) Schema() tools.ToolSchema {
    return tools.ToolSchema{
        Type: "object",
        Properties: map[string]interface{}{
            "url": map[string]interface{}{ "type": "string",
        "description": "URL to navigate to" },
        },
        Required:    []string{"url"},
        Description: "Navigate the active browser session (lazy-
        creates if none) to the given URL.",
    }
}

func (t *BrowserNavigateToolV2) Validate(params
        map[string]interface{}) error {
    u, ok := params["url"].(string)
    if !ok || u == "" { return fmt.Errorf("url is required (non-
        empty string)") }
```

```
    return nil
}

func (t *BrowserNavigateToolV2) Execute(ctx context.Context,
        params map[string]interface{}) (interface{}, error) {
    url := params["url"].(string)
    s, err := t.mgr.EnsureSession(ctx)
    if err != nil { return nil, err }
    cctx, cancel := context.WithTimeout(ctx,
        t.opts.NavigateTimeout)
    defer cancel()
    var resolvedURL, title string
    if err := s.Run(cctx,
        chromedp.Navigate(url),
        chromedp.WaitReady("body", chromedp.ByQuery),
        chromedp.Title(&title),
        chromedp.Location(&resolvedURL),
    ); err != nil {
        if errors.Is(err, context.DeadlineExceeded) { return
        nil, ErrNavigationTimeout }
        return nil, err
    }
    return map[string]interface{}{"url": resolvedURL, "title":
        title}, nil
}
```

Failing tests FIRST (Schema + Validate + RequiresApproval; Execute happy path tested via integration in T09):

```
func TestNavigateTool_Name(t *testing.T) {
    require.Equal(t, "browser_navigate",
        NewBrowserNavigateTool(nil, BrowserOptions{}).Name())
}

func TestNavigateTool_RequiresApproval_LevelEdit(t *testing.T) {
    require.Equal(t, approval.LevelEdit,
        NewBrowserNavigateTool(nil,
        BrowserOptions{}).RequiresApproval())
}

func TestNavigateTool_Validate_RequiresURL(t *testing.T) {
    tool := NewBrowserNavigateTool(nil, BrowserOptions{})
    require.Error(t, tool.Validate(map[string]interface{}{}))
    require.Error(t, tool.Validate(map[string]interface{}{"url":
        ""}))
    require.Error(t, tool.Validate(map[string]interface{}{"url":
        42}))
    require.NoError(t, tool.Validate(map[string]interface{}
        {"url": "https://example.com"}))
```

```
}

func TestNavigateTool_Schema_RequiresURL(t *testing.T) {
    sch := NewBrowserNavigateTool(nil, BrowserOptions{}).Schema()
    require.Contains(t, sch.Required, "url")
}
```

Subject: `feat(P2-F23-T04): browser_navigate tool - lazy session-create + WaitReady + 30s timeout (TDD).`

Task 5: snapshot_tool.go (TDD)

Files: new `helix_code/internal/tools/browser/snapshot_tool.go`, new `helix_code/internal/tools/browser/snapshot_tool_test.go`.

```
type BrowserSnapshotTool struct { mgr *BrowserManager; opts
        BrowserOptions }

func (t *BrowserSnapshotTool) Name() string { return
        "browser_snapshot" }
func (t *BrowserSnapshotTool) RequiresApproval()
        approval.ApprovalLevel { return approval.LevelReadOnly }

func (t *BrowserSnapshotTool) Schema() tools.ToolSchema {
    return tools.ToolSchema{
        Type: "object",
        Properties: map[string]interface{}{
            "mode": map[string]interface{}{
                "type": "string", "enum": []string{"html",
        "text"},
                "description": "Snapshot mode: 'html' returns
        OuterHTML; 'text' returns visible body text. Default
        'html'.",
            },
        },
        Required: []string{},
    }
}

func (t *BrowserSnapshotTool) Validate(params
        map[string]interface{}) error {
    if m, ok := params["mode"].(string); ok && m != "" && m !=
        "html" && m != "text" {
        return fmt.Errorf("mode must be 'html' or 'text', got
        %q", m)
    }
    return nil
}
```

```
func (t *BrowserSnapshotTool) Execute(ctx context.Context,
        params map[string]interface{}) (interface{}, error) {
    mode := "html"
    if m, ok := params["mode"].(string); ok && m != "" { mode =
        m }
    s, err := t.mgr.RequireSession()
    if err != nil { return nil, err }
    var content, pageURL, pageTitle string
    var actions []chromedp.Action
    switch mode {
    case "html":
        actions = []chromedp.Action{
            chromedp.OuterHTML("html", &content,
        chromedp.ByQuery),
            chromedp.Location(&pageURL),
            chromedp.Title(&pageTitle),
        }
    case "text":
        actions = []chromedp.Action{
            chromedp.Text("body", &content,
        chromedp.NodeVisible, chromedp.ByQuery),
            chromedp.Location(&pageURL),
            chromedp.Title(&pageTitle),
        }
    }
    if err := s.Run(ctx, actions...); err != nil { return nil,
        err }
    truncated := false
    if len(content) > MaxSnapshotBytes {
        content = content[:MaxSnapshotBytes]; truncated = true
    }
    return Snapshot{URL: pageURL, Title: pageTitle, Mode: mode,
        Content: content, Truncated: truncated}, nil
}
```

Failing tests FIRST:

```
func TestSnapshotTool_Name(t *testing.T) {
    require.Equal(t, "browser_snapshot",
        (&BrowserSnapshotTool{}).Name())
}
func TestSnapshotTool_RequiresApproval_LevelReadOnly(t
        *testing.T) {
    require.Equal(t, approval.LevelReadOnly,
        (&BrowserSnapshotTool{}).RequiresApproval())
}
func TestSnapshotTool_Validate_Mode(t *testing.T) {
    tool := &BrowserSnapshotTool{}
```

```
    require.NoError(t, tool.Validate(map[string]interface{}{}))
    require.NoError(t, tool.Validate(map[string]interface{}
        {"mode": "html"}))
    require.NoError(t, tool.Validate(map[string]interface{}
        {"mode": "text"}))
    require.Error(t, tool.Validate(map[string]interface{}
        {"mode": "json"}))
}
func TestSnapshotTool_Execute_NoSession_Err(t *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    tool := &BrowserSnapshotTool{mgr: mgr}
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{})
    require.ErrorIs(t, err, ErrNoActiveSession)
}
```

Subject: `feat(P2-F23-T05): browser_snapshot tool - html/text mode + 64 KB cap + Truncated flag (TDD).`

Task 6: click_type_tools.go (TDD)

Files: new `helix_code/internal/tools/browser/click_type_tools.go`, new `helix_code/internal/tools/browser/click_type_tools_test.go`.

`browser_click`:

```
type BrowserClickTool struct { mgr *BrowserManager; opts
        BrowserOptions }

func (t *BrowserClickTool) Name() string { return
        "browser_click" }
func (t *BrowserClickTool) RequiresApproval()
        approval.ApprovalLevel { return approval.LevelEdit }
// Schema: selector required (string).
// Execute: RequireSession → context.WithTimeout(parent, 5s) →
        chromedp.Run(ctx,
//   chromedp.Click(sel, chromedp.NodeVisible, chromedp.ByQuery),
//   chromedp.Sleep(opts.ClickWaitDuration),
//   chromedp.Location(&url), chromedp.Title(&title),
// ). On context.DeadlineExceeded → return ErrSelectorNotFound.
        On success
// return {"clicked": true, "url": url, "title": title}.
```

`browser_type`:

```
type BrowserTypeTool struct { mgr *BrowserManager; opts
        BrowserOptions }

func (t *BrowserTypeTool) Name() string { return "browser_type" }
```

```
func (t *BrowserTypeTool) RequiresApproval()
        approval.ApprovalLevel { return approval.LevelEdit }
// Schema: selector + text (both required strings).
// Execute: RequireSession → chromedp.Run(ctx,
//   chromedp.SendKeys(sel, text, chromedp.NodeVisible,
        chromedp.ByQuery),
// ). On context.DeadlineExceeded → ErrSelectorNotFound. On
        success return {"typed": true}.
```

Failing tests FIRST (Schema/Validate/RequiresApproval; Execute happy paths covered in T10 Challenge + integration):

```
func TestClickTool_RequiresApproval_LevelEdit(t *testing.T) {
    require.Equal(t, approval.LevelEdit,
        (&BrowserClickTool{}).RequiresApproval())
}
func TestClickTool_Validate_RequiresSelector(t *testing.T) {
    tool := &BrowserClickTool{}
    require.Error(t, tool.Validate(map[string]interface{}{}))
    require.Error(t, tool.Validate(map[string]interface{}
        {"selector": ""}))
    require.NoError(t, tool.Validate(map[string]interface{}
        {"selector": "#b"}))
}
func TestTypeTool_RequiresApproval_LevelEdit(t *testing.T) {
    require.Equal(t, approval.LevelEdit,
        (&BrowserTypeTool{}).RequiresApproval())
}
func TestTypeTool_Validate_RequiresSelectorAndText(t *testing.T)
        {
    tool := &BrowserTypeTool{}
    require.Error(t, tool.Validate(map[string]interface{}
        {"selector": "#in"}))
    require.Error(t, tool.Validate(map[string]interface{}
        {"text": "hi"}))
    require.NoError(t, tool.Validate(map[string]interface{}
        {"selector": "#in", "text": "hi"}))
}
func TestClickTool_Execute_NoSession_Err(t *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    tool := &BrowserClickTool{mgr: mgr}
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{"selector": "#b"})
    require.ErrorIs(t, err, ErrNoActiveSession)
}
```

Non-obvious call: chromedp's `NodeVisible` blocks until the selector becomes visible OR the parent ctx deadline fires. We wrap a 5 s sub-timeout to disambiguate "selector not found" from

“navigation slow”; without the sub-timeout, a missing selector hangs until the user’s parent ctx cancels.

Subject: `feat(P2-F23-T06): browser_click + browser_type tools - NodeVisible + 5s selector timeout + ClickWaitDuration settle (TDD).`

Task 7: screenshot_tool.go (TDD)

Files: new `helix_code/internal/tools/browser/screenshot_tool.go`, new `helix_code/internal/tools/browser/screenshot_tool_test.go`.

```
type BrowserScreenshotToolV2 struct { mgr *BrowserManager; opts
        BrowserOptions }

var pngMagic = []byte{0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A,
        0x0A}

func (t *BrowserScreenshotToolV2) Name() string { return
        "browser_screenshot" }
func (t *BrowserScreenshotToolV2) RequiresApproval()
        approval.ApprovalLevel { return approval.LevelReadOnly }

func (t *BrowserScreenshotToolV2) Execute(ctx context.Context,
        params map[string]interface{}) (interface{}, error) {
    fullPage := false
    if v, ok := params["full_page"].(bool); ok { fullPage = v }
    s, err := t.mgr.RequireSession()
    if err != nil { return nil, err }
    var buf []byte
    var action chromedp.Action
    if fullPage {
        action = chromedp.FullScreenshot(&buf, 90)
    } else {
        action = chromedp.CaptureScreenshot(&buf)
    }
    if err := s.Run(ctx, action); err != nil { return nil, err }
    if int64(len(buf)) > t.opts.ScreenshotMaxBytes {
        // Fallback to viewport-only.
        if !fullPage { return nil, ErrScreenshotTooLarge }
        buf = buf[:0]
        if err := s.Run(ctx, chromedp.CaptureScreenshot(&buf));
        err != nil { return nil, err }
        if int64(len(buf)) > t.opts.ScreenshotMaxBytes { return
        nil, ErrScreenshotTooLarge }
    }
    if len(buf) <= 8 || !bytes.Equal(buf[:8], pngMagic) {
```

```
            return nil, fmt.Errorf("browser_screenshot: chromedp
                returned non-PNG bytes (len=%d)", len(buf))
        }
        cfg, err := png.DecodeConfig(bytes.NewReader(buf))
        if err != nil { return nil, fmt.Errorf("browser_screenshot:
            invalid PNG: %w", err) }
        path := s.NextScreenshotPath()
        if err := os.MkdirAll(filepath.Dir(path), 0700); err != nil
            { return nil, err }
        if err := os.WriteFile(path, buf, 0600); err != nil { return
            nil, err }
        info, err := os.Stat(path)
        if err != nil { return nil, err }
        if info.Size() <= 1024 {
            return nil, fmt.Errorf("browser_screenshot: file too
                small (%d bytes)", info.Size())
        }
        return ScreenshotResult{Path: path, Bytes: info.Size(),
            Width: cfg.Width, Height: cfg.Height}, nil
}
```

Failing tests FIRST (PNG-magic verification; happy-path bytes via fixture):

```
func TestScreenshotTool_RequiresApproval_LevelReadOnly(t
        *testing.T) {
    require.Equal(t, approval.LevelReadOnly,
        (&BrowserScreenshotToolV2{}).RequiresApproval())
}
func TestScreenshotTool_Execute_NoSession_Err(t *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    tool := &BrowserScreenshotToolV2{mgr: mgr, opts:
        OptionsFromEnv()}
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{})
    require.ErrorIs(t, err, ErrNoActiveSession)
}
func TestScreenshotTool_PNGMagic_Bytes(t *testing.T) {
    // Direct test of the magic-bytes constant.
    require.Equal(t, []byte{0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A,
        0x1A, 0x0A}, pngMagic)
}
// Integration test in tests/integration/browser_test.go covers
        the full
// chromedp → file → PNG-DecodeConfig path against a real
        chromium subprocess.
```

Non-obvious call: PNG-magic verification happens on the in-memory `buf` BEFORE `os.WriteFile`. This catches the chromedp-returned-empty-buf bug at the source (the disk never sees junk bytes). Spec §5.2 Bluff #4 pin.

Subject: `feat(P2-F23-T07): browser_screenshot tool - PNG-magic + DecodeConfig + size>1024 + tempdir 0600 (TDD).`

Task 8: close_tool.go (TDD)

Files: new `helix_code/internal/tools/browser/close_tool.go`, new `helix_code/internal/tools/browser/close_tool_test.go`.

```
type BrowserCloseToolV2 struct { mgr *BrowserManager }

func (t *BrowserCloseToolV2) Name() string { return
        "browser_close" }
func (t *BrowserCloseToolV2) RequiresApproval()
        approval.ApprovalLevel { return approval.LevelEdit }

func (t *BrowserCloseToolV2) Execute(ctx context.Context, params
        map[string]interface{}) (interface{}, error) {
    if err := t.mgr.CloseSession(); err != nil { return nil,
        err }
    return map[string]interface{}{"closed": true}, nil
}
```

Failing tests FIRST:

```
func TestCloseTool_RequiresApproval_LevelEdit(t *testing.T) {
    require.Equal(t, approval.LevelEdit,
        (&BrowserCloseToolV2{}).RequiresApproval())
}
func TestCloseTool_Validate_NoArgs(t *testing.T) {
    tool := &BrowserCloseToolV2{}
    require.NoError(t, tool.Validate(map[string]interface{}{}))
}
func TestCloseTool_Execute_NoActiveSession_NoOpSuccess(t
        *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    tool := &BrowserCloseToolV2{mgr: mgr}
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{})
    require.NoError(t,
        err) // idempotent: closing a non-existent session is
        fine
}
func TestCloseTool_Execute_RequireFailsAfter(t *testing.T) {
    mgr := newStubManager(t)
    _, _ = mgr.EnsureSession(context.Background())
    tool := &BrowserCloseToolV2{mgr: mgr}
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{})
```

```
    require.NoError(t, err)
    _, reqErr := mgr.RequireSession()
    require.ErrorIs(t, reqErr, ErrNoActiveSession)
}
```

Subject: `feat(P2-F23-T08): browser_close tool - idempotent CloseSession + post-close RequireSession fails (TDD).`

Task 9: /browser slash + main.go wiring + integration test (TDD)

Files: new `helix_code/internal/commands/browser_command.go`, new `helix_code/internal/commands/browser_command_test.go`, new `helix_code/internal/tools/browser/register.go`, modify `helix_code/cmd/cli/main.go`, new `helix_code/tests/integration/browser_test.go` (`//go:build integration`).

`register.go`:

```
func RegisterAll(reg *tools.ToolRegistry, mgr *BrowserManager)
        error {
    opts := OptionsFromEnv()
    items := []tools.Tool{
        NewBrowserNavigateTool(mgr, opts),
        &BrowserSnapshotTool{mgr: mgr, opts: opts},
        &BrowserClickTool{mgr: mgr, opts: opts},
        &BrowserTypeTool{mgr: mgr, opts: opts},
        &BrowserScreenshotToolV2{mgr: mgr, opts: opts},
        &BrowserCloseToolV2{mgr: mgr},
    }
    for _, it := range items {
        if err := reg.RegisterTool(it); err != nil {
            return fmt.Errorf("browser.RegisterAll: %s: %w",
        it.Name(), err)
        }
    }
    return nil
}
```

`main.go` additions (additive only; adjacent to F22 wiring):

```
mgr := browser.NewBrowserManager(browser.NewDefaultChromeDiscovery(),
        c.logger)
if err := browser.RegisterAll(c.toolRegistry, mgr); err != nil {
    log.Fatalf("browser: register failed: %v", err)
}
defer mgr.CloseSession() // safety net at process exit
```

```
if regErr :=
        c.commandRegistry.Register(commands.NewBrowserCommand(mgr));
        regErr != nil {
    log.Printf("browser: register slash command failed: %v",
        regErr)
}
```

`browser_command.go` per spec §3.5 (full impl). Failing tests FIRST:

```
func TestBrowserCommand_Name(t *testing.T) {
    require.Equal(t, "browser", NewBrowserCommand(nil).Name())
}
func TestBrowserCommand_Status_Inactive(t *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    cmd := NewBrowserCommand(mgr)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"status"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "active=false")
}
func TestBrowserCommand_Close_NoActiveSession_OK(t *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    cmd := NewBrowserCommand(mgr)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"close"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "closed")
}
func TestBrowserCommand_Navigate_RequiresURL(t *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    cmd := NewBrowserCommand(mgr)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"navigate"}})
    require.Error(t, err)
}
func TestBrowserCommand_UnknownSubcommand_Err(t *testing.T) {
    mgr := NewBrowserManager(nil, zap.NewNop())
    cmd := NewBrowserCommand(mgr)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"nope"}})
    require.Error(t, err)
}
```

Failing integration tests FIRST (real chromium gated; uses real `httptest.Server`):

```
//go:build integration

func TestBrowser_Integration_NavigateThenSnapshot_FixtureSentinel(t
        *testing.T) {
```

```
    mgr, srv, cleanup := setupBrowserSession(t)
    defer cleanup()
    nav := NewBrowserNavigateTool(mgr, OptionsFromEnv())
    snap := &BrowserSnapshotTool{mgr: mgr, opts:
        OptionsFromEnv()}
    _, err := nav.Execute(context.Background(),
        map[string]interface{}{"url": srv.URL})
    require.NoError(t, err)
    res, err := snap.Execute(context.Background(),
        map[string]interface{}{"mode": "html"})
    require.NoError(t, err)
    s := res.(Snapshot)
    require.Greater(t, len(s.Content), 100)
    require.Contains(t, s.Content, "FIXTURE_LOADED_42")
    require.Equal(t, "F23-FIXTURE", s.Title)
}

func TestBrowser_Integration_ClickMutatesDOM(t *testing.T) { /*
        per spec §5.2 Bluff #2 */ }
func TestBrowser_Integration_TypeIntoInput(t *testing.T)   { /*
        per spec §6.2 */ }
func TestBrowser_Integration_Screenshot_PNGMagic(t *testing.T)
        { /* per spec §5.2 Bluff #4 */ }
func TestBrowser_Integration_Close_RequireFailsAfter(t
        *testing.T) { /* per spec §5.2 Bluff #5 */ }
func TestBrowser_Integration_ConcurrentEnsureSession_SamePointer(t
        *testing.T) { /* per spec §6.2 */ }
func TestBrowser_Integration_HeadedMode_OptIn(t *testing.T) {
    t.Setenv(EnvVarHeadedMode, "true")
    mgr, _, cleanup := setupBrowserSession(t); defer cleanup()
    _, _ = mgr.EnsureSession(context.Background())
    require.True(t, mgr.Status().Headed)
}
```

Subject: `feat(P2-F23-T09): /browser slash + main.go wiring + browser.RegisterAll + integration test (real-chromium).`

Task 10: Challenge harness 7 phases + close-out + push 4 remotes non-force

Files: new `helix_code/tests/integration/cmd/p2f23_challenge/main.go`, new `challenges/p2-f23-cline-browser-tool/CHALLENGE.md`, new `challenges/p2-f23-cline-browser-tool/run.sh`.

Harness phases (per spec §6.3):

1. **PHASE-A: NAVIGATE-AND-SNAPSHOT (always runs; gated on chromium)** — local `httptest.Server` with sentinel `FIXTURE_LOADED_42`; `browser_navigate` +

`browser_snapshot(mode=html)`; assert (i) Snapshot.Content contains sentinel, (ii) `len(Content) > 100`, (iii) Snapshot.URL equals srv.URL (or starts with), (iv) Snapshot.Title equals `F23-FIXTURE`.

2. **PHASE-B: SNAPSHOT-MODE-TEXT (always runs)** — `browser_snapshot(mode=text)`; assert (i) content does NOT contain `<p>` (it's plain text), (ii) content contains `FIXTURE_LOADED_42`.
3. **PHASE-C: CLICK-MUTATES-DOM (always runs)** — pre-click snapshot has `UNCLICKED`; `browser_click("#b")`; post-click snapshot (after ClickWaitDuration) has `CLICKED_42` AND does NOT have `UNCLICKED`.
4. **PHASE-D: TYPE-INTO-INPUT (always runs)** — `browser_type("#in", "HELIX_42")`; assert post-snapshot HTML contains `value="HELIX_42"` OR (alternatively) JS-eval reads back the input value.
5. **PHASE-E: SCREENSHOT-PNG-MAGIC (always runs)** — `browser_screenshot(full_page=false)`; assert (i) returned path exists, (ii) `os.Stat().Size() > 1024`, (iii) first 8 bytes equal `0x89 0x50 0x4E 0x47 0x0D 0x0A 0x1A 0x0A`, (iv) `image/png.DecodeConfig` succeeds.
6. **PHASE-F: CLOSE-TEARS-DOWN (always runs)** — `browser_close()`; subsequent `browser_snapshot()` returns `errors.Is(err, ErrNoActiveSession)`. Positive evidence the manager actually torn down.
7. **PHASE-G: CONCURRENT-SESSION-SHARING (always runs)** — 5 goroutines call `mgr.EnsureSession` simultaneously; assert all 5 return the same `*BrowserSession` pointer.

Output skeleton ends with:

```
SUMMARY: PHASE-A=4/4 PASS; PHASE-B=2/2 PASS; PHASE-C=3/3 PASS; PHASE-D=2/2
         PHASE-E=4/4 PASS; PHASE-F=2/2 PASS; PHASE-G=2/2 PASS
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Skip is permitted ONLY when chromium absent and MUST emit `SKIP-OK: #P2-F23 chromium not available`. Anti-bluff smoke clean check appended. Verbatim output captured into `07_phase_2_evidence.md`. Dual commit (Challenges submodule + meta-repo bump).

`challenges/p2-f23-cline-browser-tool/run.sh` mirrors F19/F20/F21/F22 structure: `cd HelixCode && go run ./tests/integration/cmd/p2f23_challenge/main.go`.

Close-out — tick all 10 items in PROGRESS, advance PROGRESS focus from F23 to "Phase 2 of CLI-Agent Fusion programme: F23 closed; F24 next candidate". Run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  internal/tools/browser internal/commands/browser_command.go &&
        echo BLUFF || echo clean
go test -count=1 ./internal/tools/browser/...
go test -count=1 ./internal/commands/...
go test -count=1 -tags=integration ./tests/integration/... #
        gated on chromium
go mod tidy
git diff --exit-code go.mod  # MUST be no-op (zero new deps)
git diff --exit-code go.sum  # MUST be no-op
```

Cross-compile check:

```
cd HelixCode && GOOS=linux GOARCH=amd64 go build -o /tmp/
        helixcode-linux-amd64 ./cmd/server
ls -la /tmp/helixcode-linux-amd64
```

Commit `chore(P2-F23-T10): close out feature 23 — Cline Browser Tool`. Push 4 remotes non-force (`origin`, `helixdev/github`, `vasic-digital/upstream`, `gitlab` per programme conventions). Request explicit user authorisation at this step (CONST-043).

PROGRESS.md milestone entry (verbatim):

```
- 2026-05-07 — Feature 23 (Cline Browser Tool) closed. 10 task commits (T0
  Real, end-to-end 6-tool browser-automation suite modelled on cline:
  browser_navigate / browser_snapshot / browser_click / browser_type /
  browser_screenshot / browser_close. Atomic-pointer BrowserManager;
  lazy-create on first navigate; sync.Once close. Headless default;
  HELIXCODE_BROWSER_HEADED=true opt-in. Screenshots written to
  $XDG_DATA_HOME/helixcode/browser/screenshots/<session>/<n>.png with
  PNG-magic + DecodeConfig + size>1024 verification. F21 RequiresApproval
  per-tool: navigate/click/type/close → LevelEdit; snapshot/screenshot
  → LevelReadOnly. /browser slash (status/navigate/close). NO cobra.
  Coexists with existing BrowserLaunchTool multi-browser API.
  Composes with F21 approval (gates LevelEdit tools), F22 autocommit
  (no spurious commits — browser tools have no path param), F13 LSP
  trigger (no-op for browser tools), F04 worktree (browser per
  CLI-instance). Zero new external deps (chromedp v0.15.1 + cdproto
  already direct). Never pushes (CONST-043). [7-phase Challenge evidence
  summary; gated on chromium availability — SKIP-OK if not on PATH].
```

Subject: `chore(P2-F23-T10): close out feature 23 — Cline Browser Tool`.

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + options (§3.3 + §10), T03 manager + session (§3.3 + §3.6), T04 navigate (§3.4 row 1), T05 snapshot (§3.4 row 2), T06 click + type (§3.4 rows 3-4), T07 screenshot (§3.4 row 5 + §5.2 Bluff #4), T08 close (§3.4 row 6 + §5.2 Bluff #5), T09 /browser slash + main.go wiring + integration (§3.5 + §4 + §6.2), T10 Challenge 7 phases (§6.3 + §5.2) + close-out.
2. **TDD:** every code task starts with failing tests. Types test pins constants byte-for-byte. Options test covers env-var case sensitivity. Manager test uses a stub `sessionFactory` to avoid spawning chromium in unit tests; integration tests use the production factory and gate on chromium availability. Six per-tool tests (T04-T08) assert Schema/Validate/RequiresApproval + no-session error paths; happy paths run under the integration test (T09) and Challenge (T10) against real chromium.
3. **Type consistency:** `Snapshot`, `ScreenshotResult`, `ManagerStatus`, `BrowserOptions`, `BrowserSession`, `BrowserManager`, `BrowserNavigateToolV2`, `BrowserSnapshotTool`, `BrowserClickTool`, `BrowserTypeTool`, `BrowserScreenshotToolV2`, `BrowserCloseToolV2`, `BrowserCommand`, sentinel errors (`ErrNoActiveSession`, `ErrChromiumNotFound`, `ErrNavigationTimeout`, `ErrSelectorNotFound`, `ErrScreenshotTooLarge`), constants (`EnvVarHeadedMode`,

`MaxSnapshotBytes`, `MaxScreenshotBytes`, `pngMagic`), command name `browser`, env var `HELIXCODE_BROWSER_HEADED` — all match across spec §3 and plan T02-T10.

4. **Zero new external deps:** chromedp v0.15.1 + cdproto are already direct in `helix_code/go.mod`. T10's verification step asserts `git diff --exit-code go.mod go.sum` is no-op. Stdlib only beyond that (image/png, net/http/httptest, sync/atomic, etc.).
5. **Anti-bluff (§5.2):** Challenge has SEVEN phases (A-G), all run on chromium availability; SKIP-OK only on chromium absence. Every phase records positive evidence: HTTP-fixture sentinel equality (PHASE-A/B), DOM-mutation byte differential (PHASE-C), input-value byte readback (PHASE-D), PNG-magic + DecodeConfig + size>1024 (PHASE-E), `errors.Is(ErrNoActiveSession)` post-close (PHASE-F), pointer equality across N=5 goroutines (PHASE-G). Byte-evidence mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.
6. **CONST-042:** browser tools NEVER log full page contents at INFO level. The committer's logger logs only the URL (caller input — already known to caller), the snapshot byte length, and the screenshot file path. A unit test scans `internal/tools/browser/*.go` for `logger\.Info\(.*\b(content|html|page|body|snapshot|outerHTML|innerText)\b` matches and FAILs on any hit (excluding test files). Per-tool descriptions and telemetry NEVER include user-typed text from `browser_type`. Screenshot files are mode `0600`.
7. **CONST-043:** F23 emits zero `git push` commands and the `BrowserSession` does NOT call any git operations. T10's close-out push to 4 remotes requires explicit user authorisation per push.
8. **CONST-033:** F23 emits no shell commands. chromium subprocess is managed by chromedp's `NewExecAllocator`, not by `os/exec`. No suspend/reboot/halt commands.
9. **F21 integration:** per-tool `RequiresApproval()` is mapped per spec §3.6. `LevelReadOnly` (snapshot, screenshot) bypass the approval gate; `LevelEdit` (navigate, click, type, close) gate behind `ModeAutoEdit` or higher. `ModeSuggest` denies `LevelEdit` browser tools, returning `ErrApprovalRequired` immediately without spawning chromium. Integration test in T09 covers both directions.
10. **F22 autocommit integration:** F22's per-tool path-derivation table doesn't include `browser_*` tool names → falls through to nil paths; F22's committer's `git status --porcelain` check returns "no changes" because browser tools don't touch the working tree → `Skipped` with `Reason: "no changes"`. Net effect: no spurious commits. Integration test in T09 asserts `git log -1 --format=%H` is unchanged after a `browser_navigate` call.
11. **F13 LSP integration:** F13's path-derivation returns nil for `browser_*` tools (no file paths), so the LSP trigger is a no-op. No interference.
12. **F04 worktree integration:** browser tools don't operate on the filesystem (other than the screenshot tempdir); worktree isolation irrelevant. Default: shared per-CLI-instance, like F22's autocommitter.
13. **Non-obvious call: coexistence with existing `BrowserLaunchTool` etc.** (recorded in spec §11 #1). Existing multi-browser API stays. F23 lands single-session API. T09's `RegisterAll` registers F23 tools under `browser_navigate`/etc. names; existing ones renamed `_legacy` if disambiguation is needed.
14. **Non-obvious call: atomic-pointer for current** (recorded in spec §11 #2). Lock-free read on every tool call; mutex serialises only EnsureSession/CloseSession lifecycle transitions.
15. **Non-obvious call: WaitReady("body") after Navigate** (recorded in spec §11 #3). Without it, OuterHTML reads empty pre-load shell.
16. **Non-obvious call: NodeVisible on Click + SendKeys** (recorded in spec §11 #4). Without it, clicks fire on hidden elements producing fake-success.
17. **Non-obvious call: PNG-magic verification BEFORE writing to disk** (recorded in spec §11 #5). Catches chromedp-empty-buf bug at the source.

18. **Non-obvious call: sync.Once on Close** (recorded in spec §11 #6). Idempotent close + tempdir RemoveAll.
19. **Non-obvious call: per-session tempdir under $XDG_DATA_HOME** (recorded in spec §11 #7). Persistence + close-time cleanup.
20. **Non-obvious call: headed via env (not CLI flag)** (recorded in spec §11 #8). Persistent across re-launches via shell profile.
21. **Non-obvious call: HELIXCODE_BROWSER_HEADED case-insensitive `true`-only** (recorded in spec §11 #9). Typos default to safe headless.
22. **Non-obvious call: per-tool RequiresApproval differs across the six tools** (recorded in spec §11 #10). Reads bypass approval; writes gate.
23. **Non-obvious call: browser_navigate is LevelEdit, NOT LevelRun** (recorded in spec §11 #11). chromium is owned by chromedp, not exposed as a shell tool; double-sandboxing breaks chromedp.
24. **Non-obvious call: browser_close is LevelEdit, NOT LevelReadOnly** (recorded in spec §11 #12). Close drops in-flight page state; user prompted before tear-down.
25. **Non-obvious call: 5s sub-timeout on click/type for selector-not-found** (recorded in spec §5.2 + plan T06). Disambiguates selector-miss from slow nav.
26. **Non-obvious call: image/png.DecodeConfig on screenshot bytes** (recorded in spec §11 #18). Lightweight (reads only IHDR chunk); positive evidence the bytes are a valid PNG, beyond magic check.
27. **Third Phase 2 feature:** F23 is the third Phase 2 feature after F21 + F22. T01 advances PROGRESS.md from "F22 closed; F23 next candidate" to "F23 in flight"; appends F23 evidence header to existing `07_phase_2_evidence.md` (created in F21-T01, extended by F22-T01).